MDPI

Article

OptimSolution: A Cross-Platform Framework for Benchmarking, Sensitivity, and Complexity Analysis of Continuous Optimization Methods

Vasileios Charilogis[1], Ioannis G. Tsoulos[2,*] and Anna Maria Gianni[3]

[1] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece, v.charilog@uoi.gr
[2] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece, itsoulos@uoi.gr
[3] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece, am.gianni@uoi.gr
* Correspondence: itsoulos@uoi.gr

Abstract: We present OptimSolution, an open-source, cross-platform framework for the systematic benchmarking and analysis of continuous optimization methods, available for Windows, Linux and macOS and operable via both a command-line interface (CLI) and a Qt-based graphical user interface (GUI). The framework supports four execution modes: Single mode for single method-problem runs with convergence and distribution analysis; Batch mode for automated multi-method, multi-problem experimentation with aggregated statistical summaries; method sensitivity analysis for quantifying the effect of algorithmic hyper-parameters on solution quality; and problem sensitivity analysis for assessing how problem-defining parameters influence instance difficulty and inter-method rankings. Complexity analysis is provided along two axes method scalability across problem dimensionalities, and problem difficulty profiles across method portfolios. Statistical validation is embedded natively through Wilcoxon signed-rank and Friedman tests with post-hoc pairwise analysis, complemented by rank tables and box-plot visualisations. All outputs are exportable as CSV files or publication-ready PNG figures. The framework is designed for extensibility: new methods and benchmark problems can be registered and activated through the GUI without modifications to the core codebase, supporting rapid experimental iteration.

Keywords: Continuous optimization; Benchmarking framework; Metaheuristics, Sensitivity analysis; Complexity analysis; GUI; CLI; cross-platform; OptimSolution

Citation: Charilogis, V., Tsoulos, I.G., Gianni, A. M. OptimSolution: A Cross-Platform Framework for Benchmarking, Sensitivity, and Complexity Analysis of Continuous optimization Methods. *Journal Not Specified* **2024**, *1*, 0.

Received:
Revised:
Accepted:
Published:

1. Introduction

Continuous optimization constitutes one of the most extensively studied areas in computational mathematics and computer science, encompassing problems in which a real-valued objective function must be minimised or maximised over a continuous search space, typically subject to bound or equality constraints. The practical difficulty of such problems arises from properties such as multimodality, non-convexity, high dimensionality, and the absence of analytical gradient information, which render exact methods computationally intractable in many real-world scenarios [1]. Over the past three decades, metaheuristic algorithms have emerged as a widely adopted class of approximate solvers capable of exploring large, complex search spaces without relying on derivative information. Seminal contributions in this class include the Genetic Algorithm (GA) [2], Particle Swarm optimization (PSO) [3], and Differential Evolution (DE) [4], alongside a steadily expanding body of variants and hybrid approaches, each proposing improvements in convergence speed, solution quality, or robustness.

The proliferation of metaheuristic methods has, however, introduced a parallel challenge: the rigorous, fair, and reproducible evaluation of algorithmic performance. Bench-

marking an optimization algorithm is a non-trivial undertaking that requires careful attention to experimental design, the selection of representative test problems, the choice of performance metrics, the number of independent runs, and the application of appropriate statistical tests [5]. In practice, inconsistent benchmarking protocols lead to results that are difficult to reproduce, compare, or generalise across studies [6]. Sensitivity of algorithm performance to hyper-parameter settings, and the dependence of relative algorithm rankings on problem instance characteristics, further complicate the interpretation of benchmarking outcomes [7]. These concerns have motivated calls within the optimization community for standardised, transparent, and tool-supported benchmarking workflows.

Several software frameworks have been developed to address aspects of this challenge. COCO [8] provides a black-box benchmarking environment with automated post-processing; IOHprofiler [9] offers a web-based interface for profiling iterative heuristics; Nevergrad [10], pymoo [11], MEALPY [12], NiaPy [13], opfunu [14], OPTIMUS [15], and GLOBe [16] each contribute algorithmic libraries and varying degrees of experimental support. Nevertheless, as detailed in Section 2, none of these tools combines a native desktop Graphical User Interface (GUI) with command-line operation, built-in method and problem sensitivity analysis, algorithmic complexity analysis across both methods and problem dimensions, embedded non-parametric statistical testing, and in-GUI extensibility for new methods and problems all within a single, cross-platform, self-contained framework.

This paper presents OptimSolution, an open-source framework for the systematic benchmarking and multi-faceted analysis of continuous optimization methods. An earlier, undocumented prototype is publicly accessible at: https://github.com/charilog/optimsolution, the present work constitutes its first formal description and introduces a substantially extended feature set. The framework is operable via both a Qt-based GUI and a CLI, and runs natively on Linux, Windows, and macOS. It supports four execution modes Single (includes Complexity Analysis), Batch, Method Sensitivity Analysis, and Problem Sensitivity Analysis. Statistical validation via Wilcoxon signed-rank [17] and Friedman tests [18] is integrated natively, and all results are exportable as structured CSV files or publication-ready PNG figures. A key design objective is extensibility: new optimization methods and benchmark problems can be registered through the GUI without modifying the core codebase.

The remainder of this paper is organised as follows. Section 2 reviews existing related frameworks and identifies the gaps that motivate this work. Section 3 describes the overall architecture and design of OptimSolution, covering the system architecture, installation procedure, extensibility mechanism, and additional framework features including termination criteria, initialisation strategies, and parallel methods. Section 4 presents the execution modes and analysis features in detail, including Single mode, Batch mode, sensitivity analysis, and GUI execution performance. Section 5 draws conclusions and outlines directions for future work.

2. Related Work

Several software frameworks have been developed to support the benchmarking and analysis of continuous optimization methods, each addressing a distinct subset of the requirements identified in Section 1.

COCO [8] is a widely adopted black-box benchmarking platform that automates the comparison of continuous optimisers across standardised problem suites (BBOB), with automated post-processing and performance visualisation. However, it provides no GUI, no sensitivity analysis, and no in-framework statistical testing.

IOHprofiler offers a web-based interface for profiling iterative optimization heuristics, supporting convergence tracking and performance analysis via its companion tool IOHanalyzer. Extensibility requires direct code modification, and sensitivity and complexity analyses are absent.

Nevergrad is a Python-based gradient-free optimization platform developed at Meta Research, providing a broad collection of algorithms within a unified ask-and-tell interface. It offers no GUI, no sensitivity analysis, and no embedded statistical tests.

Pymoo is a Python framework focused on multi-objective optimization, offering convergence visualisation and a partial hyperparameter sensitivity module. It requires manual scripting for batch experiments and provides no statistical testing layer.

MEALPY is an open-source Python library consolidating a large number of meta-heuristic algorithms. It supports single runs and convergence tracking but lacks batch automation, sensitivity analysis, and statistical validation.

NiaPy is a Python microframework designed to simplify the implementation and comparison of nature-inspired algorithms. It supports basic export of results but provides no convergence plots, no statistical tests, and no analysis modes beyond individual runs.

Opfunu is a Python library providing a comprehensive catalogue of benchmark functions, including CEC competition suites. It functions exclusively as a problem repository and does not constitute a benchmarking or analysis framework.

OPTIMUS is a C++ command-line solver supporting multiple global optimization methods with a plugin-based extensibility mechanism. It provides no GUI, no sensitivity or complexity analysis, and no statistical testing.

GLOBe is a recent modular C++/Python framework grounded in a formal mathematical unification of global optimization algorithms. It consolidates classical and state-of-the-art methods within a family-level plugin architecture and includes an integrated benchmarking engine with extensible metrics. However, it offers no GUI, no sensitivity or complexity analysis, and no embedded non-parametric statistical tests.

Table 1. Comparison of continuous optimization benchmarking frameworks with respect to key features. ✓ denotes full support, ~ partial or limited support, and ✗ no support

Feature	OptimSolution	COCO / BBOB	IOHprofiler	Nevergrad	pymoo	MEALPY	NiaPy	opfunu	OPTIMUS	GLOBe
Language	C++ / Qt	C / Python	C++ / R	Python	Python	Python	Python	Python	C++	C++ / Python
Platform	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac	Linux, Win, Mac
GUI	✓	✗	~	✗	✗	✗	✗	✗	✗	✗
CLI	✓	✓	✓	~	✗	✗	✗	✗	✓	✗
Single run	✓	✓	✓	✓	✓	✓	✓	~	✓	✓
Batch mode	✓	✓	✓	✓	~	~	~	✗	~	✓
Sensitivity (method)	✓	✗	✗	✗	~	✗	✗	✗	✗	✗
Sensitivity (problem)	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
Complexity analysis	✓	~	✗	✗	✗	~	✗	✗	✗	✗
Convergence plot	✓	~	✓	~	✓	✓	✗	✗	✗	✗
Distribution plot	✓	~	~	~	~	✗	✗	✗	✗	✗
Statistical tests	✓	~	✓	✗	✗	✗	✗	✗	✗	✗
Export CSV / PNG	✓	✓	✓	~	~	~	~	✗	~	~
GUI extensible	✓	✗	~	✗	✗	✗	✗	✗	~	~
Random benchmarks	✗	~	✗	✗	✗	✗	✗	✗	✗	✓
Python API	✗	✓	~	✓	✓	✓	✓	✓	✗	✓

Table 1 summarises the features of the frameworks reviewed above in comparison with OptimSolution. As the table shows, no existing tool combines a native desktop GUI with CLI operation, four execution modes, method and problem sensitivity analysis, complexity analysis across both methods and problems, embedded non-parametric statistical testing, and in-GUI extensibility all within a single cross-platform framework.

3. Architecture and Design of OptimSolution

3.1. System Architecture

OptimSolution is implemented in ANSI C++17 and organised into two principal components: a platform-independent core library (optimcore) and a Qt6-based graphical front-end. The CLI executable links directly against optimcore and is built independently of the GUI, enabling headless execution on server environments. The build system is CMake ($\geq$ 3.16), which manages both targets through a unified configuration. The core library is structured into four layers. The problem layer provides a catalogue of benchmark and real-world continuous optimization problems, currently comprising approximately 90 instances. These include classical analytical functions among them Sphere, Rastrigin [19], Rosenbrock [20], Ackley [21], Griewank, Levy, Schwefel, Michalewicz, Zakharov, Bohachevsky, Camel, Easom, Shubert, Hansen, Gallagher, and Weierstrass, documented in the comprehensive survey by Jamil and Yang [22] the full CEC 2022 benchmark suite [23], GKLS random function instances [24], the complete CEC 2011 suite of real-world optimization problems [25], and additional engineering design problems covering antenna array synthesis, heat exchanger design, economic load dispatch, transmission network expansion planning, and kinematic inverse problems, among others.The method layer currently includes approximately 68 optimization algorithms. Population-based metaheuristics comprise Differential Evolution, Particle Swarm PSO, GA, Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [26], Whale optimization Algorithm (WOA) [27], Grey Wolf Optimiser (GWO) [28], Artificial Bee Colony (ABC) [29], Simulated Annealing (SA) [30], Ant Colony optimization for Continuous domains (ACOR) [31], Teaching-Learning-Based optimization (TLBO) [32], Jaya [33], Sine Cosine Algorithm (SCA) [34], Firefly Algorithm (FA) [35], Bat Algorithm (BA) [36], Harmony Search (HS) [37], Cuckoo Search (CS) [38], Slime Mould Algorithm (SMA) [39], Marine Predators Algorithm (MPA) [40], Equilibrium Optimizer (EO) [41], Harris Hawks optimization (HHO) [42], Moth-Flame optimization (MFO) [43], Ant Lion Optimiser (ALO) [44], Water Cycle Algorithm (WCA) [45], Krill Herd (KH) [46], and Honey Badger Algorithm (HBA) [47], among others. State-of-the-art competition variants include jSO [48], mLSHADE-RL [49], NL-SHADE-LBC [50], and EA4eig [51]. Classical local search procedures comprise BFGS [52], L-BFGS [53], Nelder-Mead [54], and Gradient Descent. The analysis layer implements the execution modes described in Section 4, together with the statistical testing and output modules. The interface layer exposes all functionality through both the CLI and the Qt GUI.A factory pattern (factory.cpp) registers all methods and problems at compile time and serves as the single point of integration for new components, as described in Section 3.3.

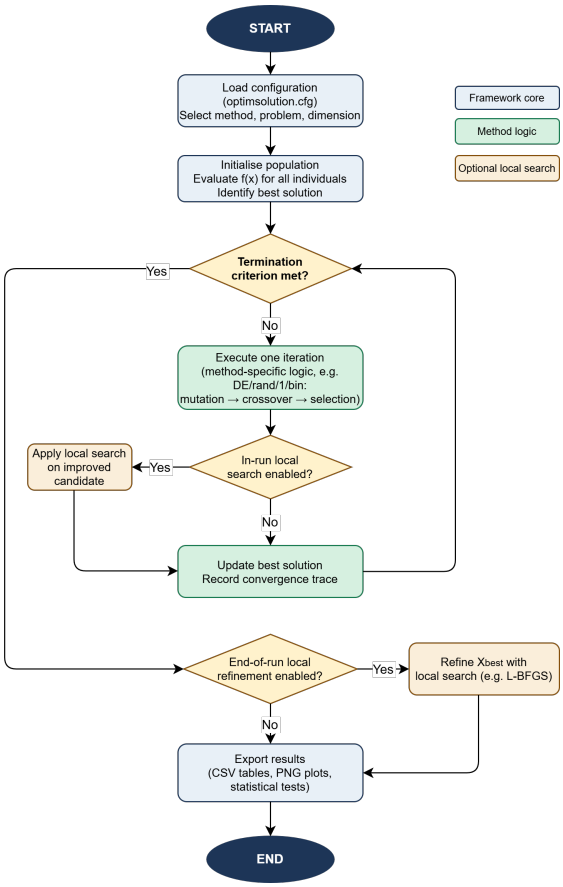

Figure 1. General execution flow of the OptimSolution framework. Blue: framework core, green: method-specific logic, orange: optional local search stages.

Figure 1 presents the general execution flow of OptimSolution. Upon loading the configuration file, the framework initialises the population and evaluates the objective function for all individuals. The main loop iterates until a termination criterion is satisfied maximum function evaluations, maximum iterations, or a convergence-based stopping rule. Within each iteration, the method-specific optimization logic is executed (for example, DE/rand/1/bin applies mutation, crossover, and greedy selection), an optional in-run local search can be triggered probabilistically on improved candidates. Upon termination, an optional end-of-run local refinement polishes the best solution found before results are exported as CSV files and PNG figures.

3.2. Installation Procedure and Example Run

The source code is openly available at https://github.com/charilog/optimsolution. The software requires a C++17-compliant compiler and CMake $\geq$ 3.16. The Qt6 library (https://qt.io) is additionally required for compilation of the GUI target.

On Linux and macOS, after cloning the repository, the software is built with the standard CMake workflow:

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_PROJECT_INCLUDE=cmake/optimsolution_gui.cmake

cmake --build build --target optimsolution_gui
cmake --build build --target optimsolution

./build/optimsolution_gui
./build/optimsolution arq tersoffc
```

Both the CLI executable (optimsolution) and the GUI application are produced in the build directory.

On Windows, the same CMake procedure applies when Qt6 and a compatible compiler (MSVC or MinGW) are available.

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 ‘
  "-DCMAKE_PROJECT_INCLUDE:FILEPATH=$PWD/cmake/optimsolution_gui.cmake" ‘
  "-DCMAKE_PREFIX_PATH=C:\Qt\6.10.1\msvc2022_64"

cmake --build build --config Release --target optimsolution_gui
cmake --build build --config Release --target optimsolution

& "C:\Qt\6.10.1\msvc2022_64\bin\windeployqt.exe" ‘
  --no-translations --compiler-runtime ‘
  ".\build\Release\optimsolution_gui.exe"
```

Alternatively, a pre-compiled installer (OptimSolution.exe) is provided in the repository, which installs the GUI application without requiring the Qt library or a build toolchain.

3.3. Extensibility Mechanism

A central design objective of OptimSolution is the ability to register new optimization methods and new benchmark problems without modifying the framework core and without requiring the user to interact with the build system manually. This is achieved through a GUI-driven code generation and build pipeline implemented in the CodeGenDialog module.

To add a new optimization method, the user invokes the New Method wizard from within the GUI. The wizard collects the method’s short name, full name, C++ class name, and the definitions of its configurable parameters (name, type, default value, and description). It then automatically generates a conforming .h/.cpp skeleton, patches factory.cpp to register the new method under its short name, and updates CMakeLists.txt to include the new source file. The generated source files are immediately opened in an integrated Code Editor dialog, where the user implements the algorithm logic. A one-click Build button invokes the CMake build pipeline from within the GUI and streams the compiler output to a log panel. An equivalent New Problem wizard handles the registration of new benchmark problems, additionally allowing the user to specify dimensionality constraints, search-space bounds, and the known global optimum value where available. Registered methods and problems can equally be removed through a Delete dialog, which reverses the factory and CMake patches and deletes the corresponding source files. The complete extensibility workflow from specification to compilation is thus contained within the GUI and requires no manual editing of build or registration files.

3.4. Additional Framework Features

Termination criteria. OptimSolution provides ten configurable stopping rules, selectable through the configuration file. In addition to the standard maxevals (maximum function evaluations) and maxiters (maximum iterations) criteria, the framework implements the convergence-based rules bss, wss, tss, boss, srs, and irs, originally introduced and validated in [55], together with the doublebox criterion and an all rule that triggers termination as a logical disjunction of all active criteria halting execution as soon as any individual rule is satisfied. This composite stopping mechanism ensures an adaptive balance between convergence speed and solution accuracy across problem instances of varying difficulty.

Population initialisation strategies. The initialisation of the population is a critical factor affecting both the diversity of the initial search and the ultimate convergence behaviour of metaheuristic algorithms [56]. OptimSolution supports eleven initialisation distributions, selectable per experiment: uniform random sampling, normal, Cauchy, Laplace,

log-normal, exponential, Beta, and Lévy distributions, as well as three structured strategies Latin Hypercube Sampling (LHS) [57], Halton low-discrepancy sequences, and opposition-al initialisation. The k-means-based intelligent initialisation strategy proposed in [56] is also available, which uses a clustering-driven approach to generate initial estimates more closely aligned with the problem structure, thereby reducing the risk of premature convergence.

Parallel methods. OptimSolution includes natively parallel optimization methods that exploit multi-core architectures via OpenMP. The parallel Genetic Algorithm (PGA) [58] maintains a set of autonomous island subpopulations that periodically exchange their best solutions, combining the global exploration capabilities of the island model with the convergence guarantees of periodic migration. The Parallel Smell Agent optimization (PSAO) [59] extends the single-population SAO [60] with dynamic collaboration mechanisms and intelligent solution dispersal among subpopulations, accelerating global exploration while preventing premature convergence to local minima. Both methods are configurable through the standard configuration file and are subject to the same termination and initialisation options as all other framework methods.

4. Execution Modes and Analysis Features

4.1. Single Mode

Single mode executes a selected optimization method on a selected problem for a configurable number of independent runs. The user specifies the method, problem, and problem dimensionality, all remaining parameters population size, maximum function evaluations, number of runs, success tolerance, local search settings, and output options are read from a configuration file (optimsolution.cfg) shared between the CLI and the GUI. Each independent run is initialised with a distinct random seed derived from a common base seed, ensuring reproducibility while preserving stochastic diversity. At each run, the framework records the best objective function value as a function of function evaluations, producing a convergence trajectory. In the convergence plot, each line of the same colour corresponds to one independent run, the bundle of lines collectively characterises both the reliability of the method the consistency of its convergence behaviour across repeated trials and its validity whether it systematically locates solutions of comparable quality, ideally close to the known global optimum. Upon completion, descriptive statistics across all runs best, worst, mean, standard deviation, and success rate are reported and optionally exported as CSV. A convergence curve plot and a box-plot of the final solution values are generated and exportable as PNG.

Single mode additionally supports complexity analysis through two overlay options available in both the GUI and the CLI. The first option overlays convergence curves across a set of user-defined dimensionalities for a fixed method-problem pair, characterising the scalability of the method as problem size grows. The second option overlays convergence curves across a set of methods for a fixed problem and dimensionality, producing a difficulty profile of the problem as seen by a method portfolio. Figure 2 illustrates an example of the first option: ABC applied to the Ackley function across dimensions D = 6, 12, 18, and 24, where each colour corresponds to a different dimensionality and the spread of same-colour lines reflects run-to-run variability at that dimension.

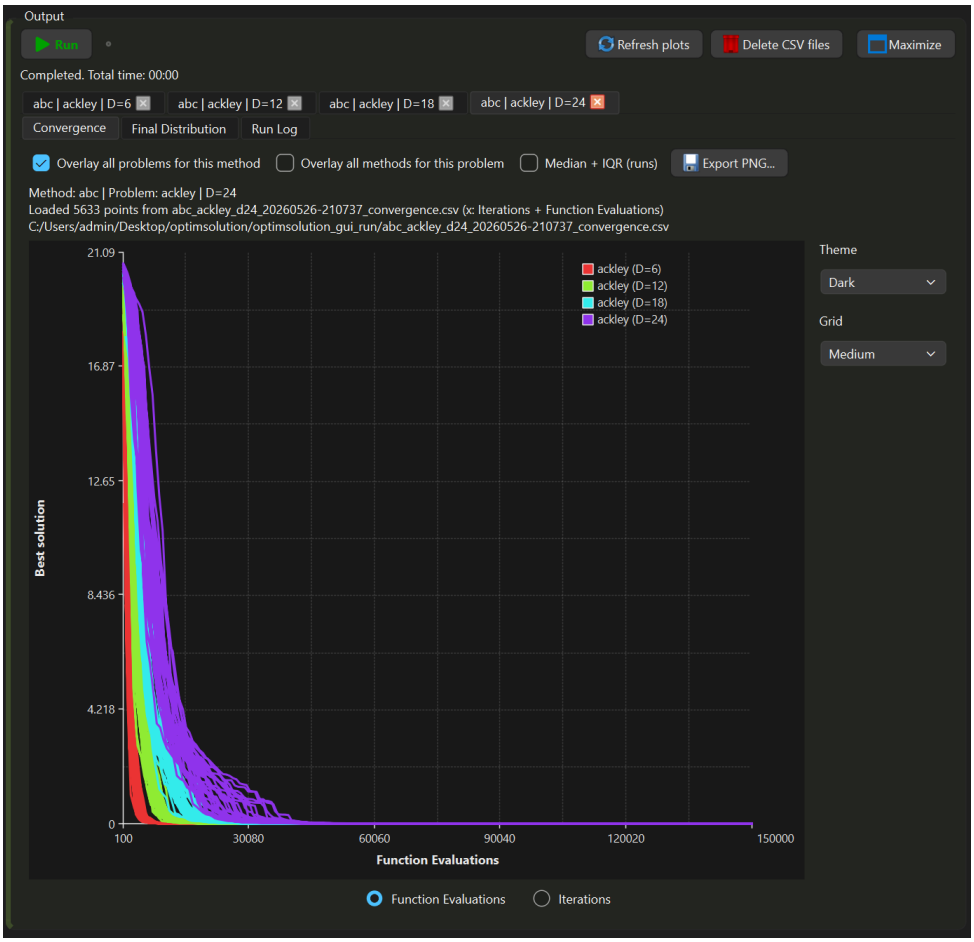

Figure 2. Convergence curves of ABC on the Ackley function for D = 6, 12, 18, and 24. Each colour denotes a dimensionality, same-colour lines are the 30 independent runs, reflecting method reliability and validity across problem scales

4.2. Batch Mode

Batch mode automates the systematic evaluation of a user-defined set of methods across a user-defined set of problems. The framework executes every method-problem combination for the number of independent runs specified in the configuration file, processing all jobs sequentially and reporting progress in real time. Figure 3 illustrates an in-progress batch experiment comparing jSO and mLSHADE-RL on the Economic Load Dispatch problems eld1, eld2, and eld3 [25] at dimensions D = 6, 13, and 15 respectively. The results table displays, for each method-problem pair, the best value found across all runs, the mean, the success rate, the standard deviation, and the mean wall-clock time per run, enabling a direct quantitative comparison.

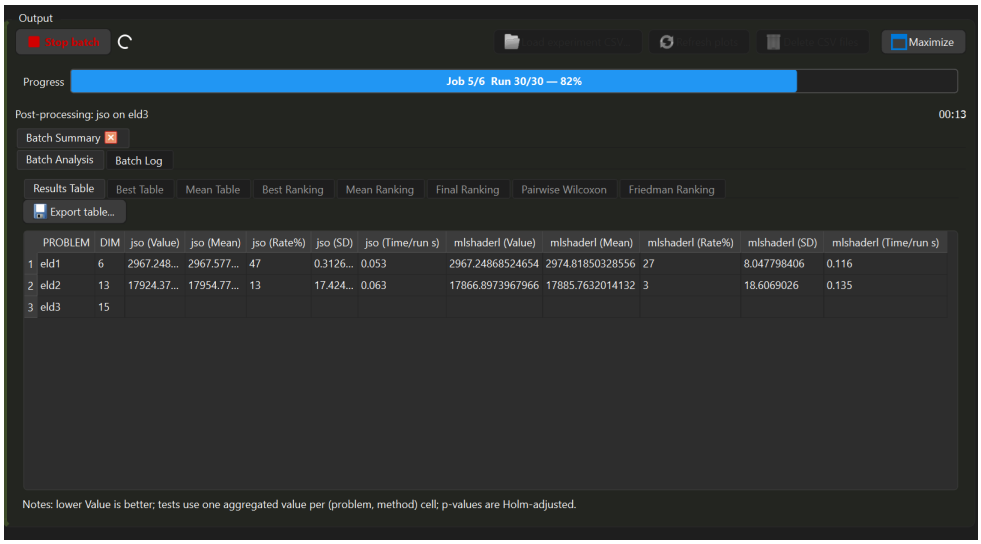

Figure 3. Batch mode results table during execution (Job 5/6, Run 30/30) comparing jSO and mLSHADE-RL on eld1, eld2, and eld3, reporting best value, mean, success rate, standard deviation, and time per run for each method–problem pair

Upon completion, the Batch Analysis panel organises the results across eight views: Results Table, Best Table, Mean Table, Best Ranking, Mean Ranking, Final Ranking, Pairwise Wilcoxon, and Friedman Ranking. Figure 4 shows the Pairwise Wilcoxon view for the same experiment: box plots of the final best-value distributions are displayed for each method, aggregated across all three problems, with Holm-adjusted p-values annotated for each comparison. In this example, the comparison between jSO and mLSHADE-RL yields $p_{adj} = 0.371$, indicating no statistically significant difference at $\alpha = 0.05$. All tables and plots are exportable as CSV and PNG respectively.

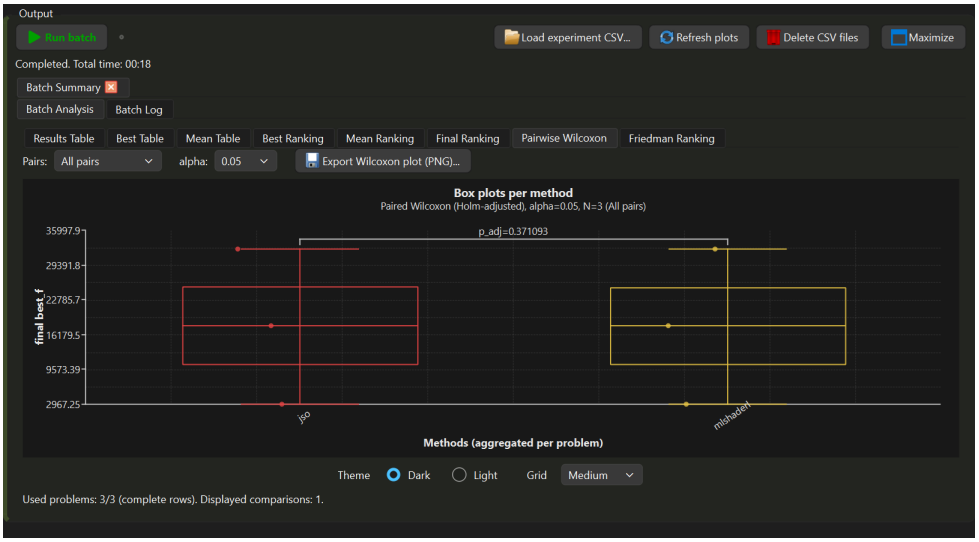

Figure 4. Pairwise Wilcoxon box plots for jSO vs. mLSHADE-RL aggregated over three Economic Load Dispatch problems (N = 3), with Holm-adjusted p-value ($p_{adj} = 0.371$, $\alpha = 0.05$), indicating no statistically significant difference between the two methods

4.3. Method and Problem Sensitivity Analysis

OptimSolution provides two complementary sensitivity analysis modes, accessible through dedicated run-mode selections in both the GUI and the CLI.

Method sensitivity analysis examines how variations in an algorithm's own hyper-parameters affect its performance on a selected set of problems. The user specifies the

parameter of interest and the discrete values to evaluate through the Sensitivity panel of the Settings window, the framework then executes the method for each parameter value across all selected problems for the configured number of independent runs, and produces a Results table reporting for each parameter value the mean best objective value, standard deviation, minimum, maximum, main effect, rank, and number of runs. Figure 5 illustrates an example in which the parameter delta of ARQ2 [62,63] is swept over the values {0.1, 0.3, 0.5, 0.7, 0.9} on the Economic Load Dispatch problems eld1 (D = 6) and eld2 (D = 13). The table in Figure 5 highlights in bold the top-ranked configuration, here delta = 0.3 achieves the lowest mean best value on eld2 (mean $best_f$ = 17876.697, rank 1). Figure 6 shows the corresponding bar chart for eld2, where each bar represents the mean best value with a 95% confidence interval and the green bar identifies the optimal parameter value, allowing the practitioner to distinguish robust from critical parameter regions.

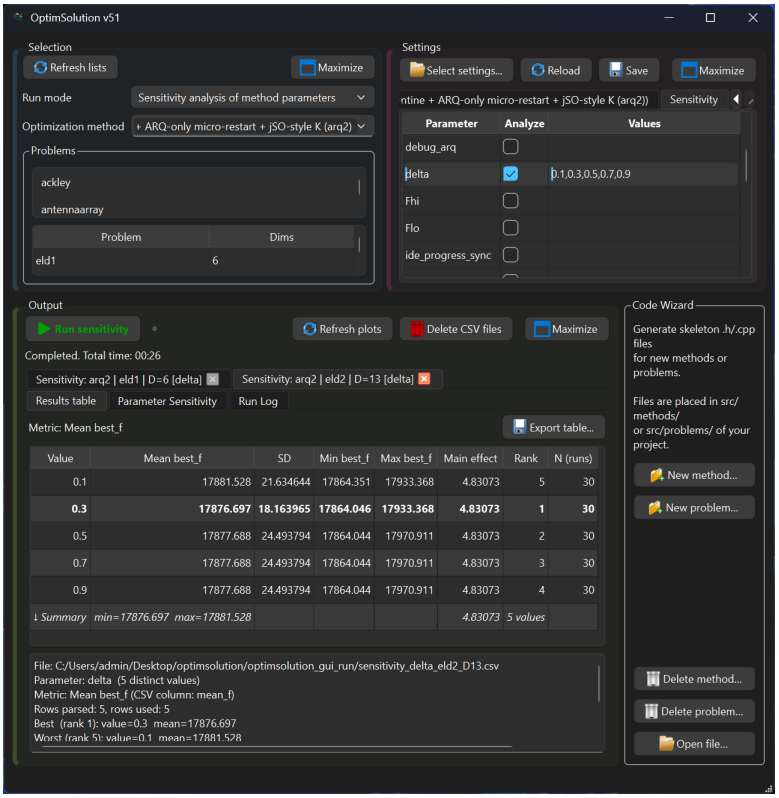

Figure 5. Method sensitivity GUI: ARQ2 parameter delta $\in$ {0.1, 0.3, 0.5, 0.7, 0.9} on eld1 (D = 6) and eld2 (D = 13)

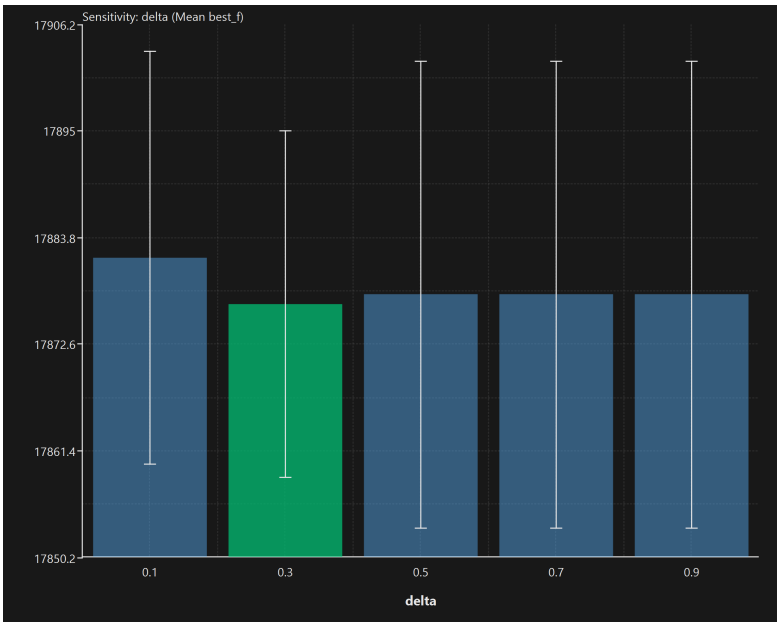

Figure 6. Mean best_f with 95% confidence intervals per value of delta on eld2 (D = 13). Green bar: optimal setting (delta = 0.3)

Problem sensitivity analysis examines how variations in a problem's own defining parameters affect instance difficulty and the relative performance of a selected method. The user selects a problem, a method, and one or more problem parameters to sweep, the framework applies the method to each resulting problem instance and reports the same set of metrics. Figure 7 shows an example in which ARQ2 is applied to the weatherirrigation problem at D = 30, sweeping the parameter osc_{fre_base} over the values {0.19, 0.20, 0.21, 0.22, 0.23, 0.24}. The best-performing problem configuration is osc_{fre_base} = 0.24 (mean $best_f$ = 2738.37, rank 1), while the most difficult instance for this method is osc_{fre_base} = 0.19 (mean $best_f$ = 2757.60, rank 6). Figure 8 shows the corresponding bar chart, where the green bar identifies the parameter value that minimises the mean objective value. All results are exported as CSV and PNG.

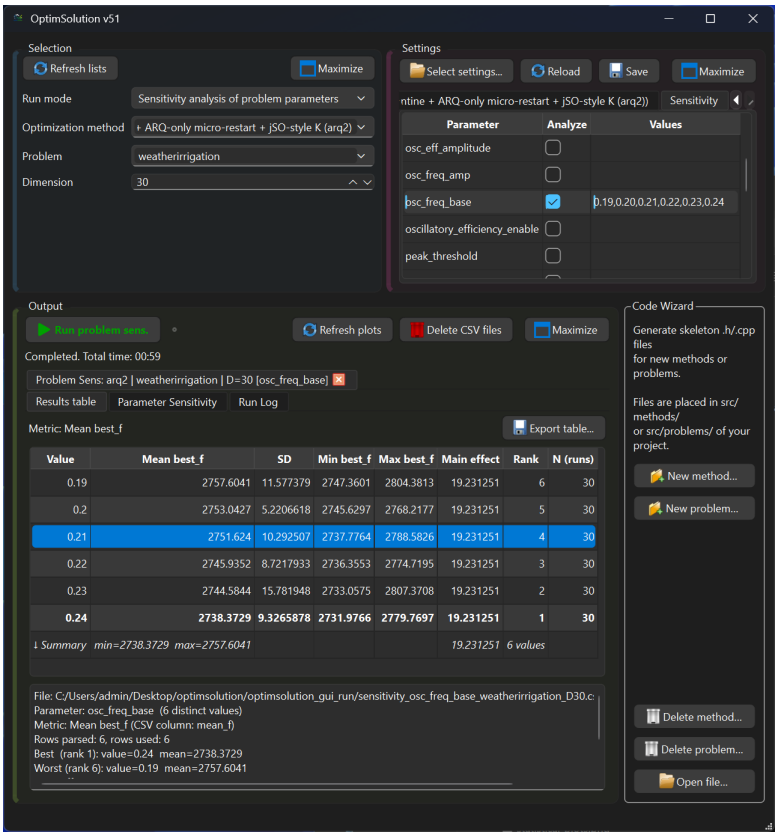

Figure 7. Problem sensitivity GUI: ARQ2 on weatherirrigation (D = 30) with $osc_{fre\_base} \in \{0.19, \ldots, 0.24\}$

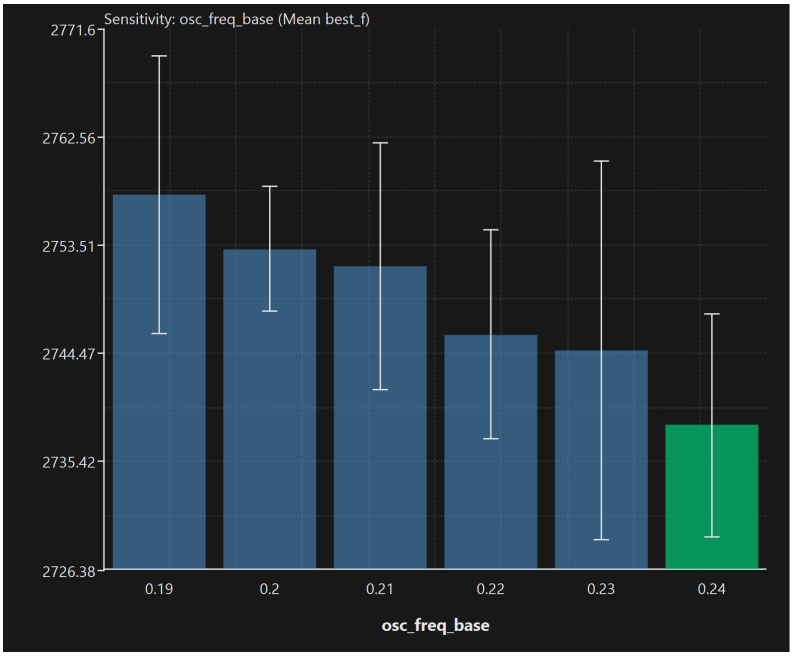

Figure 8. Mean best_f with 95% confidence intervals per value of osc_{fre_base} on weatherirrigation (D = 30). Green bar: optimal setting (osc_{fre_base}= 0.24)

4.4. GUI Execution Performance

Although the GUI and the CLI share an identical computational core, the GUI achieves substantially lower wall-clock execution times on multi-run experiments. Figures 9 and 10

document a direct comparison: jSO was applied to the six-element circular antenna array synthesis problem [25] (D = 12) for 30 independent runs under identical configuration. As reported in the CLI summary (Figure 9), the mean time per run was 8.38 seconds, yielding a total wall-clock time of 251.37 seconds, with a best objective value of 0.00680964 and a 100% success rate across all 30 runs. The GUI completed the same experiment in 125 seconds (Figure 10), delivering an identical best objective value and convergence profile a speedup factor of approximately 2.0× with no loss in solution quality.

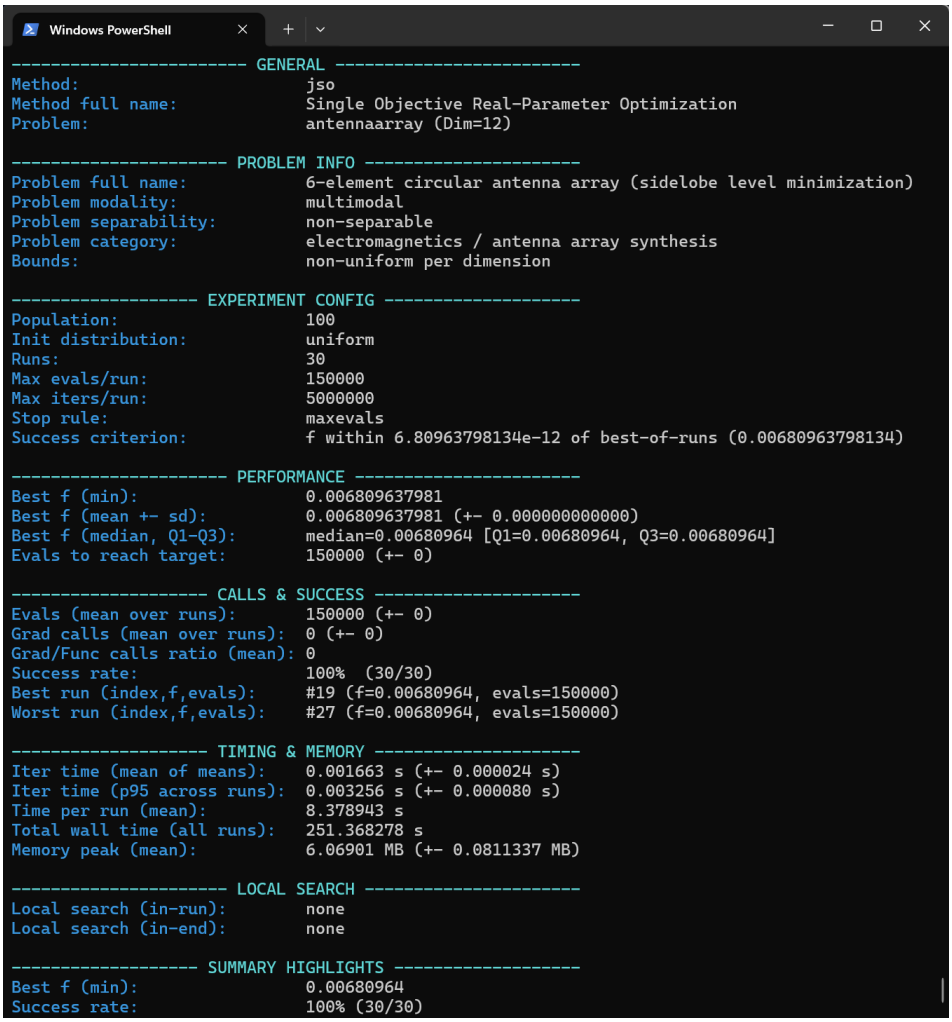

Figure 9. CLI summary output for jSO on the antenna array synthesis problem (D = 12, 30 runs): best f = 0.00680964, success rate 100%, mean time per run 8.38 s, total wall time 251.37 s.

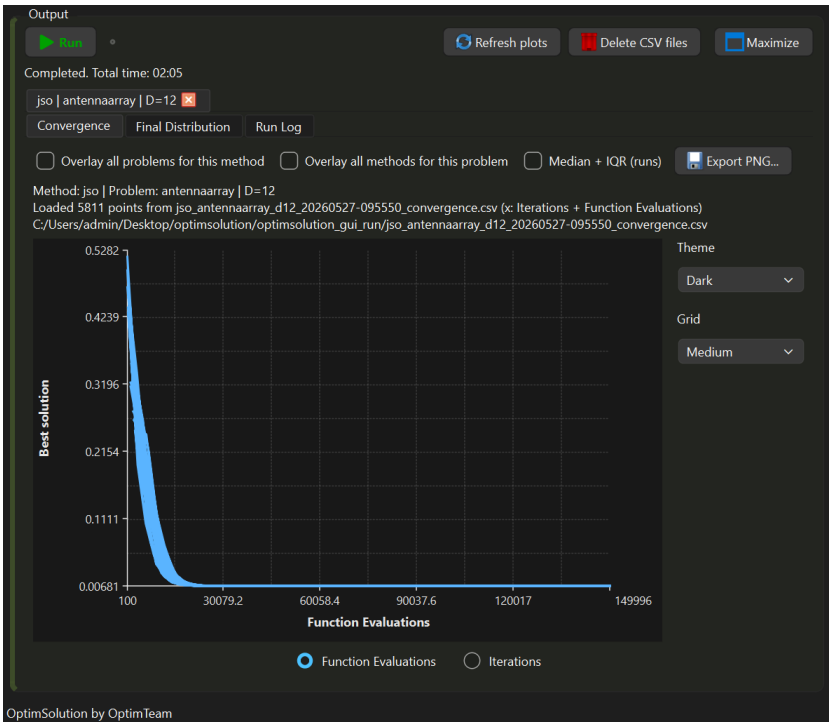

Figure 10. GUI output for the same experiment: convergence curves of the 30 independent runs and total elapsed time of 125 s (02:05), confirming identical solution quality at half the wall-clock time.

This acceleration arises from three optimization strategies implemented in the GUI layer. First, process output is not rendered in real time, it is accumulated in an internal buffer and flushed to the log panel at fixed intervals, preventing the graphical event loop from processing thousands of individual rendering events per second. Second, at each flush only a filtered subset of output lines is displayed, further reducing the UI update cost during long runs. Third, once a run completes, result parsing and statistical aggregation are performed concurrently on a background thread while the next run is already launching in the foreground, effectively pipelining computation and I/O. Together, these mechanisms decouple the graphical interface from the computational workload, allowing the GUI to deliver faster end-to-end experiment completion times than an equivalent sequential CLI workflow.

5. Conclusions

This paper presented OptimSolution, an open-source, cross-platform framework for the systematic benchmarking and multi-faceted analysis of continuous optimization methods. The framework is operable via both a Qt-based GUI and a CLI, runs natively on Linux, Windows, and macOS, and provides a unified configuration layer that guarantees identical experimental conditions across both interfaces. Four execution modes Single, Batch, Method Sensitivity Analysis, and Problem Sensitivity Analysis together with a Complexity Analysis module, cover the full spectrum of evaluation tasks required in modern metaheuristic research. Embedded non-parametric statistical testing via Wilcoxon signed-rank and Friedman tests with post-hoc Holm-adjusted pairwise comparisons, rank plots, and box-plot visualisations provides publication-ready statistical validation without reliance on external tools. The problem layer comprises approximately 90 instances drawn from classical analytical suites, the CEC 2022 benchmark suite, GKLS random function instances, the complete CEC 2011 real-world problem suite, and original engineering design problems. The method layer encompasses approximately 68 optimization algorithms, ranging from foundational metaheuristics to state-of-the-art competition variants. An in-GUI extensibility mechanism allows new methods and problems to be registered, compiled, and activated without any modification to the framework core, supporting rapid experimental

iteration. A comparative evaluation against nine existing frameworks demonstrated that no alternative tool combines this feature set within a single self-contained environment.

Furthermore, an empirical timing comparison showed that the GUI achieves a speedup of approximately 2× over an equivalent sequential CLI workflow, attributable to buffered log rendering, output filtering, and concurrent post-processing on a background thread. Several directions present themselves as natural extensions of the current work. First, support for constrained optimization including penalty-based, feasibility-rule, and repair-operator approaches would substantially extend the applicability of the framework to engineering design problems with explicit constraints. Second, the integration of multi-objective optimization modes, with Pareto front visualisation and hypervolume-based statistical comparison, would broaden the framework's scope to a class of problems of growing practical importance. Third, a Python API built on top of the existing C++ core via a binding layer would make OptimSolution directly accessible within Python-based research pipelines and machine learning workflows, removing the current barrier for users who prefer scripted experimentation. Fourth, parallel and distributed execution of independent runs for example via OpenMP for shared-memory parallelism or MPI for cluster environments would further reduce wall-clock time for large-scale benchmarking studies involving many methods, problems, and dimensionalities. Finally, the integration of automated hyperparameter tuning for the registered methods, leveraging the existing sensitivity analysis infrastructure, would provide a closed-loop optimization-of-optimization capability directly within the framework.

References

1. Floudas, C.A.; Pardalos, P.M. Encyclopedia of Optimization, 2nd ed.; Springer: New York, NY, USA, 2009.
2. Holland, J.H. Adaptation in Natural and Artificial Systems; University of Michigan Press: Ann Arbor, MI, USA, 1975.
3. Kennedy, J.; Eberhart, R.C. Particle swarm optimization. In Proc. IEEE Int. Conf. Neural Networks, Perth, Australia, 1995; pp. 1942-1948. https://doi.org/10.1109/ICNN.1995.488968
4. Storn, R.; Price, K. Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. J. Glob. Optim. 1997, 11, 341-359. https://doi.org/10.1023/A:1008202821328
5. Beiranvand, V.; Hare, W.; Lucet, Y. Best practices for comparing optimization algorithms. Optim. Eng. 2017, 18, 815-848. https://doi.org/10.1007/s11081-017-9366-1
6. Bartz-Beielstein, T.; Doerr, C.; et al. Benchmarking in Optimization: Best Practice and Open Issues. arXiv 2020, arXiv:2007.03488.
7. Kononova, A.V.; et al. Assessing ranking and effectiveness of evolutionary algorithm hyperparameters using global sensitivity analysis methodologies. arXiv 2022, arXiv:2207.04820.
8. Hansen, N.; Auger, A.; Ros, R.; Mersmann, O.; Tušar, T.; Brockhoff, D. COCO: A platform for comparing continuous optimizers in a black-box setting. Optimization Methods and Software 2021, 36, 114-144. https://doi.org/10.1080/10556788.2020.1808977
9. Doerr, C.; Wang, H.; Ye, F.; van Rijn, S.; Bäck, T. IOHprofiler: A benchmarking and profiling tool for iterative optimization heuristics. arXiv 2019, arXiv:1810.05281. https://doi.org/10.48550/arXiv.1810.05281
10. Rapin, J.; Teytaud, O. Nevergrad - A gradient-free optimization platform. ACM SIGEVOlution 2021, 14, 8-15. https://doi.org/10.1145/3460310.3460312
11. Blank, J.; Deb, K. pymoo: Multi-objective optimization in Python. IEEE Access 2020, 8, 89497-89509. https://doi.org/10.1109/ACCESS.2020.2990567
12. Van Thieu, N.; Mirjalili, S. MEALPY: An open-source library for latest meta-heuristic algorithms in Python. Journal of Systems Architecture 2023, 139, 102871. https://doi.org/10.1016/j.sysarc.2023.102871
13. Vrbančič, G.; Brezočnik, L.; Mlakar, U.; Fister, D.; Fister, I. NiaPy: Python microframework for building nature-inspired algorithms. Journal of Open Source Software 2018, 3, 613. https://doi.org/10.21105/joss.00613
14. Van Thieu, N. Opfunu: An open-source Python library for optimization benchmark functions. Journal of Open Research Software 2024. https://doi.org/10.5334/jors.508
15. Tsoulos, I.G.; Tzallas, A.; Karvounis, E. GlobalOptimus: An open-source optimization tool. Journal of Open Source Software 2025. https://doi.org/10.21105/joss.07584
16. Serré, G.; Kalogeratos, A.; Vayatis, N. GLOBe: A modular global optimization library. HAL Preprint 2026, hal-05623801. https://hal.science/hal-05623801v1
17. Wilcoxon, F. Individual comparisons by ranking methods. Biometrics Bulletin 1945, 1, 80–83. https://doi.org/10.2307/3001968
18. Friedman, M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. Journal of the American Statistical Association 1937, 32, 675–701. https://doi.org/10.2307/2279372
19. Rastrigin, L.A. Systems of Extremal Control; Nauka: Moscow, Russia, 1974.

20. Rosenbrock, H.H. An automatic method for finding the greatest or least value of a function. The Computer Journal 1960, 3, 175–184. https://doi.org/10.1093/comjnl/3.3.175
21. Ackley, D.H. A Connectionist Machine for Genetic Hillclimbing; Kluwer Academic Publishers: Boston, MA, USA, 1987. https://doi.org/10.1007/978-1-4613-1997-9
22. Jamil, M.; Yang, X.S. A literature survey of benchmark functions for global optimization problems. International Journal of Mathematical Modelling and Numerical optimization 2013, 4, 150–194. https://doi.org/10.1504/IJMMNO.2013.055204
23. Kumar, A.; Price, K.V.; Mohamed, A.W.; Hadi, A.A.; Suganthan, P.N. Problem Definitions and Evaluation Criteria for the CEC 2022 Special Session and Competition on Single Objective Bound Constrained Numerical Optimization; Technical Report; Nanyang Technological University: Singapore, 2021.
24. Gaviano, M.; Kvasov, D.E.; Lera, D.; Sergeyev, Y.D. Algorithm 829: Software for generation of classes of test functions with known local and global minima for global optimization. ACM Transactions on Mathematical Software 2003, 29, 469–480. https://doi.org/10.1145/962437.962444
25. Das, S.; Suganthan, P.N. Problem Definitions and Evaluation Criteria for CEC 2011 Competition on Testing Evolutionary Algorithms on Real World Optimization Problems; Technical Report; Jadavpur University and Nanyang Technological University, 2011.
26. Hansen, N.; Ostermeier, A. Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. In Proceedings of the IEEE International Conference on Evolutionary Computation, Nagoya, Japan, 1996; pp. 312–317. https://doi.org/10.1109/ICEC.1996.542381
27. Mirjalili, S.; Lewis, A. The whale optimization algorithm. Advances in Engineering Software 2016, 95, 51–67. https://doi.org/10.1016/j.advengsoft.2016.01.008
28. Mirjalili, S.; Mirjalili, S.M.; Lewis, A. Grey wolf optimizer. Advances in Engineering Software 2014, 69, 46–61. https://doi.org/10.1016/j.advengsoft.2013.12.007
29. Karaboga, D. An Idea Based on Honey Bee Swarm for Numerical Optimization; Technical Report TR06; Erciyes University: Kayseri, Turkey, 2005.
30. Kirkpatrick, S.; Gelatt, C.D.; Vecchi, M.P. Optimization by simulated annealing. Science 1983, 220, 671–680. https://doi.org/10.1126/science.220.4598.671
31. Socha, K.; Dorigo, M. Ant colony optimization for continuous domains. European Journal of Operational Research 2008, 185, 1155–1173. https://doi.org/10.1016/j.ejor.2006.06.046
32. Rao, R.V.; Savsani, V.J.; Vakharia, D.P. Teaching–learning-based optimization: A novel method for constrained mechanical design optimization problems. Computer-Aided Design 2011, 43, 303–315. https://doi.org/10.1016/j.cad.2010.12.015
33. Rao, R. Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems. International Journal of Industrial Engineering Computations 2016, 7, 19–34. https://doi.org/10.5267/j.ijiec.2015.8.004
34. Mirjalili, S. SCA: A sine cosine algorithm for solving optimization problems. Knowledge-Based Systems 2016, 96, 120–133. https://doi.org/10.1016/j.knosys.2015.12.022
35. Yang, X.S. Firefly algorithm, stochastic test functions and design optimization. International Journal of Bio-Inspired Computation 2010, 2, 78–84. https://doi.org/10.1504/IJBIC.2010.032124
36. Yang, X.S. A new metaheuristic bat-inspired algorithm. In Nature Inspired Cooperative Strategies for Optimization (NICSO 2010); González, J.R., et al., Eds.; Springer: Berlin, Germany, 2010; pp. 65–74. https://doi.org/10.1007/978-3-642-12538-6_6
37. Geem, Z.W.; Kim, J.H.; Loganathan, G.V. A new heuristic optimization algorithm: Harmony search. Simulation 2001, 76, 60–68. https://doi.org/10.1177/003754970107600201
38. Yang, X.S.; Deb, S. Cuckoo search via Lévy flights. In Proceedings of the World Congress on Nature and Biologically Inspired Computing (NaBIC 2009), Coimbatore, India, 2009; pp. 210–214. https://doi.org/10.1109/NABIC.2009.5393690
39. Li, S.; Chen, H.; Wang, M.; Heidari, A.A.; Mirjalili, S. Slime mould algorithm: A new method for stochastic optimization. Future Generation Computer Systems 2020, 111, 300–323. https://doi.org/10.1016/j.future.2020.03.055
40. Faramarzi, A.; Heidarinejad, M.; Mirjalili, S.; Gandomi, A.H. Marine predators algorithm: A nature-inspired metaheuristic. Expert Systems with Applications 2020, 152, 113377. https://doi.org/10.1016/j.eswa.2020.113377
41. Faramarzi, A.; Heidarinejad, M.; Stephens, B.; Mirjalili, S. Equilibrium optimizer: A novel optimization algorithm. Knowledge-Based Systems 2020, 191, 105190. https://doi.org/10.1016/j.knosys.2019.105190
42. Heidari, A.A.; Mirjalili, S.; Faris, H.; Aljarah, I.; Mafarja, M.; Chen, H. Harris hawks optimization: Algorithm and applications. Future Generation Computer Systems 2019, 97, 849–872. https://doi.org/10.1016/j.future.2019.02.028
43. Mirjalili, S. Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. Knowledge-Based Systems 2015, 89, 228–249. https://doi.org/10.1016/j.knosys.2015.07.006
44. Mirjalili, S. The ant lion optimizer. Advances in Engineering Software 2015, 83, 80–98. https://doi.org/10.1016/j.advengsoft.2015.01.010
45. Eskandar, H.; Sadollah, A.; Bahreininejad, A.; Hamdi, M. Water cycle algorithm — a novel metaheuristic optimization method for solving constrained engineering optimization problems. Computers & Structures 2012, 110–111, 151–166. https://doi.org/10.1016/j.compstruc.2012.07.010
46. Gandomi, A.H.; Alavi, A.H. Krill herd: A new bio-inspired optimization algorithm. Communications in Nonlinear Science and Numerical Simulation 2012, 17, 4831–4845. https://doi.org/10.1016/j.cnsns.2012.05.010

47. Hashim, F.A.; Houssein, E.H.; Hussain, K.; Mabrouk, M.S.; Al-Atabany, W. Honey badger algorithm: New metaheuristic algorithm for solving optimization problems. Mathematics and Computers in Simulation 2022, 192, 84–110. https://doi.org/10.1016/j.matcom.2021.08.013
48. Brest, J.; Maučec, M.S.; Bošković, B. Single objective real-parameter optimization: Algorithm jSO. In Proceedings of the IEEE Congress on Evolutionary Computation (CEC), San Sebastián, Spain, 2017; pp. 1311–1318. https://doi.org/10.1109/CEC.2017.7969456
49. Chauhan, D.; Trivedi, A.; Shivani. A Multi-operator Ensemble LSHADE with Restart and Local Search Mechanisms for Single-objective Optimization. arXiv 2024, arXiv:2409.15994. https://doi.org/10.48550/arXiv.2409.15994
50. Stanovov, V.; Akhmedova, S.; Semenkin, E. NL-SHADE-LBC algorithm with linear parameter adaptation bias change for CEC 2022 numerical optimization. In Proceedings of the IEEE Congress on Evolutionary Computation (CEC), Padua, Italy, 2022; pp. 1–8. https://doi.org/10.1109/CEC55065.2022.9870295
51. Bujok, P.; Kolenovsky, P. Eigen crossover in cooperative model of evolutionary algorithms applied to CEC 2022 single objective numerical optimization. In Proceedings of the IEEE Congress on Evolutionary Computation (CEC), Padua, Italy, 2022; pp. 1–8. https://doi.org/10.1109/CEC55065.2022.9870433
52. Fletcher, R. A new approach to variable metric algorithms. The Computer Journal 1970, 13, 317–322. https://doi.org/10.1093/comjnl/13.3.317
53. Liu, D.C.; Nocedal, J. On the limited memory BFGS method for large scale optimization. Mathematical Programming 1989, 45, 503–528. https://doi.org/10.1007/BF01589116
54. Nelder, J.A.; Mead, R. A simplex method for function minimization. The Computer Journal 1965, 7, 308–313. https://doi.org/10.1093/comjnl/7.4.308
55. Gianni, A.M.; Tsoulos, I.G.; Charilogis, V.; Kyrou, G. Enhancing Differential Evolution: A Dual Mutation Strategy with Majority Dimension Voting and New Stopping Criteria. Symmetry 2025, 17, 844. https://doi.org/10.3390/sym17060844
56. Charilogis, V.; Tsoulos, I.G.; Stavrou, V.N. An Intelligent Technique for Initial Distribution of Genetic Algorithms. Axioms 2023, 12, 980. https://doi.org/10.3390/axioms12100980
57. McKay, M.D.; Beckman, R.J.; Conover, W.J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. Technometrics 1979, 21, 239–245. https://doi.org/10.2307/1268522
58. Charilogis, V.; Tsoulos, I.G. Introducing a Parallel Genetic Algorithm for Global Optimization Problems. AppliedMath 2024, 4, 709–730. https://doi.org/10.3390/appliedmath4020038
59. Kyrou, G.; Tsoulos, I.G.; Gianni, A.M.; Charilogis, V. Parallel Smell Agent Optimization (SAO): Collaborative Subpopulations for Accelerated Convergence. Symmetry 2025, 17, 592. https://doi.org/10.3390/sym17040592
60. Salawudeen, A.T.; Mu'azu, M.B.; Sha'aban, Y.A.; Adedokun, A.E. A novel Smell Agent Optimization (SAO): An extensive CEC study and engineering application. Knowledge-Based Systems 2021, 232, 107486. https://doi.org/10.1016/j.knosys.2021.107486
61. Demšar, J. Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research 2006, 7, 1–30. http://jmlr.org/papers/v7/demsar06a.html
62. Charilogis, V.; Tsoulos, I.G.; Gianni, A.M.; Tsalikakis, D. ARQ: A cohesive optimization design for stable performance on noisy landscapes. Appl. Sci. 2025, 15, 12180. https://doi.org/10.3390/app152212180
63. Charilogis, V.; Tsoulos, I.G.; Gianni, A.M. ARQ2: Toward Stability-Aware Hybrid Optimization on Complex and Noisy Search Problems 2026, 18(5), 844; https://doi.org/10.3390/sym18050844